Project Report: Infosec Demo â€“ Certificate Handshake Fix

Project Name: infosec

Owner: aqsaaaaaaaaaaaa

Repository: https://github.com/aqsaaaaaaaaaaaa/infosec

Current Branch: fix/demo-cert-handshake

Commit Hash: 7474106

Date: November 17, 2025

Executive Summary

This report documents the diagnosis and resolution of a critical certificate verification and signature handling bug in the `infosec` project's cryptographic demo. The issue prevented successful clientâ€“server handshakes due to incorrect certificate signature verification, improper public-key export, and double-hashing of signed data. All issues have been identified, fixed, and successfully tested.

Problem Statement

Initial Error

The client failed to complete the initial TLS-like handshake with the server:

```
ConnectionError: Socket closed before receiving all data
```

Traceback location: `client.py` line 78 in `recv_json()` â†’ line 65 in `recvall()` â†’ line 59

Root Cause Analysis

Investigation revealed **three distinct bugs**:

1. **Incorrect Certificate Verification (Fatal)**

• **Location:** Both `client.py` and `server.py`, function `verify_cert_signed_by_root()`

• **Issue:** Missing `asym_padding.PKCS1v15()` parameter when calling `pubkey.verify()`

• **Impact:** Certificate signature verification always raised an exception, causing the server to reject the client and close the socket immediately

• **Error seen:** `pubkey.verify(cert.signature, cert.tbs_certificate_bytes, cert.signature_hash_algorithm)` â€" incorrect signature verification interface

2. **Incorrect Public Key Export (Fatal)**

• **Location:** `server.py` line 61, `client.py` line 119 (ack signature verification)

• **Issue:** Used `x509.Encoding` and `x509.PublicFormat` which do not exist

• **Correct location:** `cryptography.hazmat.primitives.serialization` module

• **Error:** `AttributeError: module 'cryptography.x509' has no attribute 'Encoding'`

• **Impact:** Server crashed during ack signature verification, before sending the ack response to the client

3. **Double-Hash Mismatch (Non-Fatal but Critical)**

• **Location:** `client.py` line 105, `server.py` lines 138, 148

• **Issue:** Signing functions internally hash with SHA-256, but caller was pre-hashing the data before passing to sign functions

• **Example:**

- **Wrong (client):** `sig = sign_with_client(SHA256.new(signed_data).digest())`

- **Correct:** `sig = sign_with_client(signed_data)`

• **Impact:** Signature verification failed because the signed digest did not match the verified data (one level of hashing mismatch)

• **Error seen:** `Signature verification failed` on server side

Solution Overview

Fix 1: Certificate Verification â€" Add PKCS#1 v1.5 Padding

File: `demo/client.py` and `demo/server.py`

Function: `verify_cert_signed_by_root(pem_bytes)`

Before:

```
pubkey.verify(cert.signature, cert.tbs_certificate_bytes, cert.signature_hash_algorithm)
```

After:

```
from cryptography.hazmat.primitives.asymmetric import padding as asym_padding
pubkey.verify(
cert.signature,
cert.tbs_certificate_bytes,
asym_padding.PKCS1v15(),
cert.signature_hash_algorithm,
)
```

Reason: The `cryptography` library's RSA signature verification requires the padding scheme (PKCS#1 v1.5) to be explicitly specified.

Fix 2: Public Key Export â€“ Use Correct Serialization Module

File: `demo/server.py` line 61, `demo/client.py` line 119

Function: `verify_signature_with_rsa_pub()` and ack verification

Before:

```
pub_pem = pub.public_bytes(encoding=x509.Encoding.PEM,
format=x509.PublicFormat.SubjectPublicKeyInfo)
```

After:

```
from cryptography.hazmat.primitives import serialization
pub_pem = pub.public_bytes(
encoding=serialization.Encoding.PEM,
format=serialization.PublicFormat.SubjectPublicKeyInfo
)
```

Reason: `Encoding` and `PublicFormat` classes are defined in `serialization` module, not `x509`. The `x509` module handles certificate objects, not key serialization.

Fix 3: Signature Hashing â€“ Remove Double-Hash

File: `demo/client.py` line 105, `demo/server.py` lines 138, 148

Before (client):

```
signed_data = (str(seqno) + "|" + str(ts) + "|" + ct_b64).encode()
sig = sign_with_client(SHA256.new(signed_data).digest()) # WRONG: pre-hashed
```

After:

```
signed_data = (str(seqno) + "|" + str(ts) + "|" + ct_b64).encode()
```

```
sig = sign_with_client(signed_data) # CORRECT: raw data
```

Before (server transcript/ack):

```
t_sig = sign_with_server(SHA256.new(tdata).digest())
ack_sig = sign_with_server(SHA256.new(ack_bytes).digest())
```

After:

```
t_sig = sign_with_server(tdata)
ack_sig = sign_with_server(ack_bytes)
```

Reason: Both `sign_with_client()` and `sign_with_server()` internally perform SHA-256 hashing before signing. Pre-hashing the data caused a mismatch: the verifier would hash the original data (correct), but the signature was made on the hash of the hash (incorrect).

Files Modified

File	Changes	Status
`demo/client.py`	Added serialization import, fixed cert verification padding, fixed public-key export, removed double-hash	✅ Fixed
`demo/server.py`	Added serialization import, fixed cert verification padding, fixed public-key export, removed double-hash	✅ Fixed
`demo/check_certs.py`	**New file** – Verification helper to test client/server certs against root CA	✅ Created

Cryptographic Protocol Details

The demo implements a simplified secure registration protocol:

Protocol Flow

```
1. Client Hello
- Client sends: {client_cert (PEM), A (DH public key)}
- Server verifies client cert against root CA
2. Server Response
- Server sends: {B (DH public key), server_cert (PEM)}
- Client verifies server cert against root CA
- Both derive shared secret: K = SHA256(A^b mod p)[:16] [AES-128 key]
3. Encrypted Registration
- Client sends: {ct (AES-CBC(registration, iv)), sig (RSA sign), seqno, ts}
- Server verifies client signature on (seqno|ts|ct)
- Server decrypts and stores user in SQLite DB
- Server appends transcript entry and signs transcript file
4. Server Acknowledgment
```

```
- Server sends: {ack (status, ts), sig (RSA sign of ack)}
- Client verifies server signature on ack
```

Cryptographic Primitives

Component	Algorithm	Details
Key Agreement	Diffie-Hellman (DH)	RFC 3526 Group 14 (2048-bit MODP)
Session Key Derivation	SHA-256 truncation	AES-128 key = SHA256(shared_secret)[:16]
Symmetric Encryption	AES-128-CBC	PKCS#7 padding
Signature	RSA-SHA256	PKCS#1 v1.5 padding
Certificates	X.509 (self-signed chain)	Root CA â†’ Client/Server certs
Password Hash	SHA256	Salt (8 bytes) + password, stored in SQLite

Test Results

Successful Handshake Output

Server terminal:

```
Server listening on 9000
Accepted ('127.0.0.1', 57732)
Client cert verified against Root CA.
Derived AES key (hex): cec9f0e6cbab730592ed2731b583dd13
Client signature OK
Decrypted payload: {'type': 'register', 'username': 'student1', 'email': 's1@example.com', 'pwd': 'password123'}
Stored user student1
Sent ack and signed transcript
```

Client terminal:

```
Server cert verified.
Derived AES key (hex): cec9f0e6cbab730592ed2731b583dd13
Ack signature OK; ack: {'status': 'ok', 'ts': 1763352755278}
```

Key Observations

âœ… **Certificates verified successfully** â€” both client and server certs validated against root CA

âœ… **DH key agreement successful** â€” both sides derived identical AES-128 key

âœ… **Signature verification passed** â€— client registration signed correctly

âœ… **Symmetric encryption/decryption successful** â€” payload correctly encrypted and decrypted

âœ… **Server ack signature verified** â€” client verified server's ack signature

âœ… **User registration stored** â€” SQLite DB successfully saved registration

Code Quality Notes

Additional Observations

1. **Duplicate `recvall` definition in `client.py`** (lines 44â€“53)

- Both definitions are identical

- Recommendation: Remove one for cleanliness

- Impact: None (second definition shadows first, both work)

2. **Exception handling in `verify_signature_with_rsa_pub()`** (line 60 in server.py)

- Uses bare `except:` clause

- Recommendation: Catch specific exceptions (`Crypto.Signature.pkcs1_15.verify` exceptions) for better debugging

- Impact: Low â€” error is returned as False anyway

3. **No server-side error messages before socket close**

- If certificate verification fails, server closes socket without sending JSON error

- Recommendation: Send framed JSON error message before closing (improves UX)

- Impact: Low â€” current behavior is acceptable for demo

How to Run the Demo

Prerequisites

```
python -m venv .venv
```

```
.\.venv\Scripts\Activate.ps1
```

```
pip install pycryptodome cryptography
```

Generate Certificates

```
cd scripts
./gen_ca.sh
./gen_cert.sh server server.local
./gen_cert.sh client client.local
```

Run the Demo

Terminal 1 (Server):

```
cd demo
python server.py
```

Terminal 2 (Client):

```
cd demo
python client.py
```

Verify Certificates

```
cd demo
python check_certs.py
```

Expected output:

```
client.pem: VALID - signed by rootCA
server.pem: VALID - signed by rootCA
```

Artifacts Generated

After successful run:

• `demo/users.db` â€” SQLite database with registered users

• `demo/transcript.log` â€” JSON audit log of all transactions

• `demo/transcript.log.sig` â€” RSA signature of transcript file

Git Commit

Branch: `fix/demo-cert-handshake`

Commit Hash: `7474106`

Commit Message:

```
Fix certificate verification, public key export, and signature handling in demo; add
check_certs helper
```

Files committed:

• `demo/client.py` (modified)

• `demo/server.py` (modified)

• `demo/check_certs.py` (new)

Remote: https://github.com/aqsaaaaaaaaaaaa/infosec.git

Conclusion

All three critical bugs have been fixed. The clientâ€“server handshake now completes successfully, cryptographic operations verify correctly, and user registration is properly stored with signed audit trails. The demo is ready for use and testing.

Summary of Changes

Bug	Severity	Status	Fix
Missing PKCS#1 v1.5 padding in cert verification	**CRITICAL**	âœ… Fixed	Added `asym_padding.PKCS1v15()` parameter
Incorrect public key export module	**CRITICAL**	âœ… Fixed	Changed from `x509.Encoding` to `serialization.Encoding`
Double-hashing in signature operations	**HIGH**	âœ… Fixed	Removed pre-hash before calling sign functions

Report prepared by: GitHub Copilot

Date: November 17, 2025

Status: âœ… All fixes implemented and tested successfully